Evaluation and Light Customization of Large Language Models

Goals

Course Objectives and Schedule

- Learn why evaluating your LLMs is crucial for ensuring their accuracy, reliability, and ethical use.
- Compare zero-shot vs. few-shot prompting and PEFT (Parameter-Efficient Fine-Tuning) as strategies for injecting knowledge into your LLM
- Gain hands-on experience with Unit-Testing LLMs, implementing zero-shot, few-shot, and PEFT workflows in real-world LLM applications
- Explore comprehensive methods for End-to-End Evaluation of LLMs
- Master techniques for Visualizing and Acting on Evaluation Results

Why do we evaluate LLMs?

Unlocking the Power of LLMs: Why Evaluation Matters

A continuous process that validates the value of LLMs and Agentic AI

Maximizing Accuracy and Reliability

Boosting User Trust and Satisfaction == Adoption

Enhancing Safety and Compliance

Adapting to Local Languages and Contexts

Preventing AI Hallucinations

Facilitating Continuous Improvement

Making Data-Driven Decisions in LLM-Integrated App Development

Using evaluation data to drive critical technical and business decisions in LLM deployment.

- Which **LLM fits the application's scope** (e.g., GPT-4 vs. smaller models)? -> Fine-Tuning & Customization
- Determine whether to use **out-of-the-box performance** or invest in **fine-tuning**. -> Prompt Engineering & Input Control
- Decide on **prompt strategies** that yield reliable outputs. -> Deployment Strategy
- Cloud vs. on-premises, **cost vs. latency** considerations. -> Safety, Bias, & Ethical Considerations
- Evaluate **error rates** and ensure **fairness** and **compliance**. -> Continuous Learning & Monitoring
- When to update models based on **real-world performance**.

Evaluation Approaches Overview

Assessing LLM Performance: Strategies to Evaluate and Improve Your App

Controlled Evaluation – focus for this lab

Automated Benchmarking using Open-Source SDKs

- Utilize standardized datasets and industry-approved metrics
- Examples: Hugging Face's evaluation utilities, EleutherAI's lm_eval

Synthetic & Adversarial Test Cases

- Generate controlled inputs to stress-test model behavior
- Helps assess robustness and edge-case performance

Human-Centered Evaluation

Manual Review

- Experts or in-house evaluators examine outputs for nuance, clarity, and relevance
- Ideal for tasks where subtle language or tone matters

Crowdsourced or Third-Party Assessments

- Leverage feedback from a diverse group of non-experts
- Provides broader perspectives on quality and contextual appropriateness

Field Evaluation (Real-World Performance)

Production Monitoring / Live Evaluation Methods

- Deploy the model's endpoint and collect metrics from real user interactions
- Track KPIs such as response latency, error rates, and user satisfaction
- Methods include A/B testing, log analysis, and real-time feedback loops

Zoom-in: Controlled Evaluation – Tasks

Using standardized and repeatable test cases to assess model performance

Language Modeling & Generation	• Task Examples: Story completion, dialogue generation, creative writing • Aim: Test fluency, coherence, style consistency
Question Answering (QA)	• Task Examples: Factual QA, reasoning questions\ • Aim: Measure correctness and relevance
Machine Translation & Summarization	• Task Examples: Translating text between languages, summarizing long documents • Aim: Verify semantic accuracy and content retention
Text Classification & Sentiment Analysis	• Task Examples: Topic classification, sentiment detection • Aim: Evaluate prediction accuracy on labeled datasets
Conversational Context Understanding	• Task Examples: Multi-turn dialogue evaluations • Aim: Assess context retention and appropriate response generation

Zoom-in: Controlled Evaluation – Metrics

Using standardized and repeatable test cases to assess model performance

Measuring the localization quality of the model – speaking the right language…

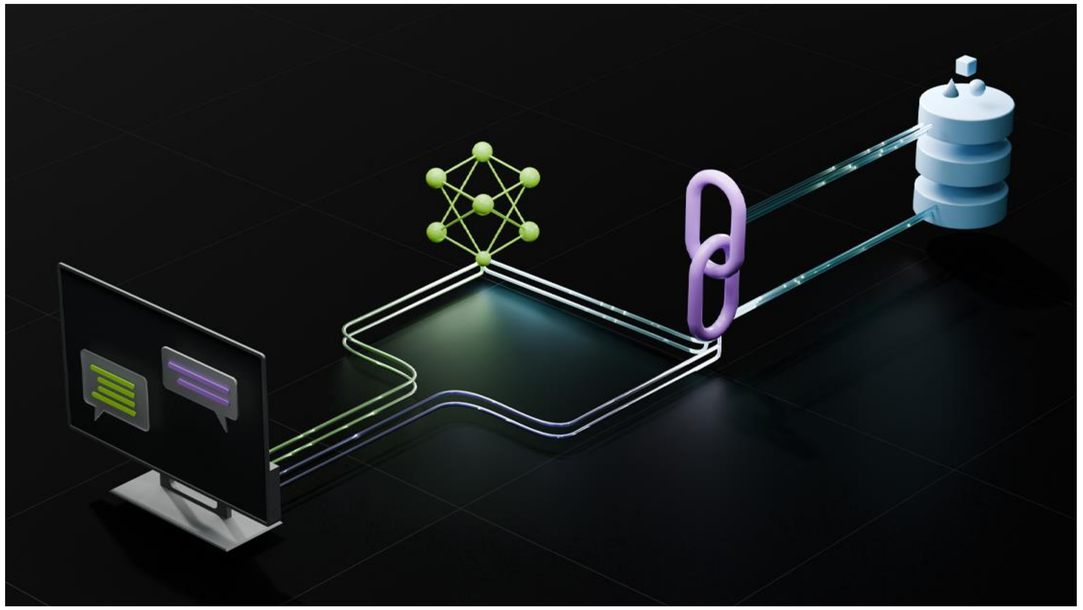
Quantitative Metrics

- Standard Accuracy Metrics:
  - **Accuracy**, Precision, **Recall**, F1 Score (e.g., for text classification and QA tasks)
- Language Modeling Metrics:
  - **Perplexity**: Measures how well a model predicts a sample
  - **BLEU/ROUGE**: Common in machine translation and summarization for n-gram overlap
- Academic Benchmark Scores:
  - **GLUE/SuperGLUE**: A collection of language understanding tasks
  - **SQuAD** Scores: For question-answering performance
  - **MMLU, GSM8K**, and Similar Benchmarks:
    – GSM8K: Focuses on quantitative reasoning and math problem-solving
    – These benchmarks provide insight into how well the model performs on **diverse**, **domain-specific reasoning tasks**

NVIDIA CONFIDENTIAL. DO NOT DISTRIBUTE.

Zoom-in: Controlled Evaluation – Metrics

Using standardized and repeatable test cases to assess model performance



Qualitative and Hybrid Metrics

- Human Judgments and LLM-as-a-Judge Evaluations:
  - Use one LLM to **assess responses** from another (or previous versions)
  - **Compare responses** on criteria such as clarity, coherence, and factual correctness
  - Examples: Pairwise comparisons, ranking outputs in open-ended tasks
- Task-Specific Quality:
  - **Coherence and Relevance**: Often rated on a Likert scale by experts
  - **Creativity and Diversity**: Subjective assessments that may be captured through crowdsourced ratings
- Model Robustness & Bias Checks:
  - **Consistency**: How consistently does a model perform across diverse test cases?
  - **Bias & Fairness Metrics**: Evaluated using specially designed prompts or datasets

Showcasing LLM Performance

How Research Papers Introduce Evaluation Results

- **Benchmark Categories:** Clearly state the types of tasks and benchmarks used.
  - *Example Phrasing:* "Our evaluation suite comprises benchmarks covering:
  - **General Language Understanding & Knowledge:** (e.g., MMLU, Hellaswag, ARC)
  - **Reasoning:** (e.g., GSM8K for math, BigBench Hard for complex reasoning)
  - **Code Generation:** (e.g., HumanEval, MBPP)
  - **Safety & Alignment:** (e.g., TruthfulQA, ToxiGen, custom red-teaming)
  - **Multilinguality, Summarization, Translation, etc.**"
- **Evaluation Settings:**
  - Specify the **prompting strategy** (e.g., zero-shot, few-shot, instruction-tuned).
    - *Example Phrasing:* "Performance is reported in few-shot settings (k=5) for most benchmarks, unless otherwise specified."
  - Clearly list the **primary metrics** used for each benchmark category.
    - *Example Phrasing:* "We use accuracy for knowledge and reasoning tasks, pass@k for coding benchmarks, ROUGE scores for summarization..."
- State which **existing models** the new LLM is being compared against.
  - *Example Phrasing:* "We compare [Model Name]'s performance against several state-of-the-art open and closed models, including [Model A, Model B, and a general category like 'other leading foundation models']."
- Papers then typically present **detailed tables** showing side-by-side comparisons of scores across numerous benchmarks.
  - The introductory text sets up these tables, guiding the reader on what to look for.
  - *Example Phrasing:* "Table X summarizes the main results across all evaluated benchmarks. We highlight instances where [Model Name] achieves new state-of-the-art performance or demonstrates significant improvements in specific areas."

Showcasing LLM Performance

How Research Papers Introduce Evaluation Results

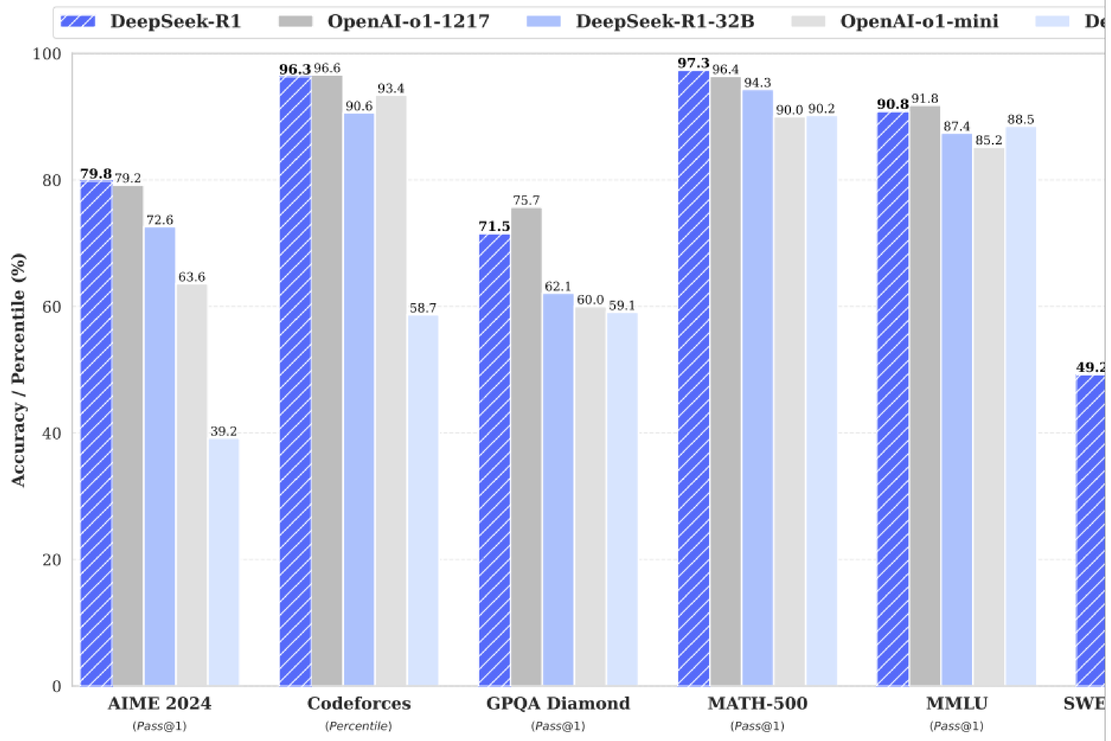


Figure 1 | Benchmark performance of DeepSeek-R1.

	Benchmark (Metric)	Claude-3.5-Sonnet-1022	GPT-4o 0513	DeepSeek V3	OpenAI o1-mini	OpenAI o1-1217	DeepSeek R1
	Architecture	-	-	MoE	-	-	MoE
	# Activated Params	-	-	37B	-	-	37B
	# Total Params	-	-	671B	-	-	671B
English	MMLU (Pass@1)	88.3	87.2	88.5	85.2	**91.8**	90.8
	MMLU-Redux (EM)	88.9	88.0	89.1	86.7	-	**92.9**
	MMLU-Pro (EM)	78.0	72.6	75.9	80.3	-	**84.0**
	DROP (3-shot F1)	88.3	83.7	91.6	83.9	90.2	**92.2**
	IF-Eval (Prompt Strict)	**86.5**	84.3	86.1	84.8	-	83.3
	GPQA Diamond (Pass@1)	65.0	49.9	59.1	60.0	**75.7**	71.5
	SimpleQA (Correct)	28.4	38.2	24.9	7.0	**47.0**	30.1
	FRAMES (Acc.)	72.5	80.5	73.3	76.9	-	**82.5**
	AlpacaEval2.0 (LC-winrate)	52.0	51.1	70.0	57.8	-	**87.6**
	ArenaHard (GPT-4-1106)	85.2	80.4	85.5	92.0	-	**92.3**
Code	LiveCodeBench (Pass@1-COT)	38.9	32.9	36.2	53.8	63.4	**65.9**
	Codeforces (Percentile)	20.3	23.6	58.7	93.4	**96.6**	96.3
	Codeforces (Rating)	717	759	1134	1820	**2061**	2029
	SWE Verified (Resolved)	**50.8**	38.8	42.0	41.6	48.9	49.2
	Aider-Polyglot (Acc.)	45.3	16.0	49.6	32.9	**61.7**	53.3
Math	AIME 2024 (Pass@1)	16.0	9.3	39.2	63.6	79.2	**79.8**
	MATH-500 (Pass@1)	78.3	74.6	90.2	90.0	96.4	**97.3**
	CNMO 2024 (Pass@1)	13.1	10.8	43.2	67.6	-	**78.8**
Chinese	CLUEWSC (EM)	85.4	87.9	90.9	89.9	-	**92.8**
	C-Eval (EM)	76.7	76.0	86.5	68.9	-	**91.8**
	C-SimpleQA (Correct)	55.4	58.7	**68.0**	40.3	-	63.7

From DeepSeek Paper - https://arxiv.org/pdf/2501.12948

Beyond the Hype: The Case for Custom LLM Evaluation

Why you should run your own evaluations

- Public benchmarks and leaderboards offer a **starting point**, but relying solely on them can be **misleading**
- Developers gain **critical advantages** by conducting their own LLM evaluations tailored to their **specific needs** –

Verify Real-World Performance on Your Specific Use Case

Uncover Domain-Specific Weaknesses and Biases

Make Informed Model Selection & Comparison

Drive Iterative Improvement & Fine-Tuning

Justify Cost-Benefit and Resource Allocation

Custom LLM evaluation moves you from relying on generalized claims to understanding **true, actionable performance** for your **unique context**, leading to better products and reduced risk.

Metric in details: MMLU Evaluation

Massive Multitask Language Understanding (MMLU) benchmark evaluates general knowledge and reasoning

Structure & Task Format:

- Multiple-Choice Questions:
  - Each question has 4–5 answer options.
  - Questions are designed to test factual accuracy, reasoning, and domain-specific understanding.
  - Diverse Task Categories:
    - Subjects range from elementary topics to advanced professional knowledge.
    - Examples: Mathematics, physics, history, law, and computer science.

- Example (History Domain):
  "Which Roman emperor initiated the construction of Constantinople as the new capital of the Roman Empire?"
  Options:
  A. Trajan
  B. Constantine
  C. Diocletian
  D. Marcus Aurelius

Metric in details: MMLU Evaluation

Massive Multitask Language Understanding (MMLU) benchmark evaluates general knowledge and reasoning

- Evaluation Process Using Logits
  - Prompting & Inference:
    – The model receives the standardized prompt including the question-and-answer choices.
    – It generates raw output scores (logits) for each option based on its internal reasoning.

- Decision & Scoring:
  – The answer with the highest probability (Option A) is selected as the model's response.
  – The response is compared with the ground truth to determine if it is correct.

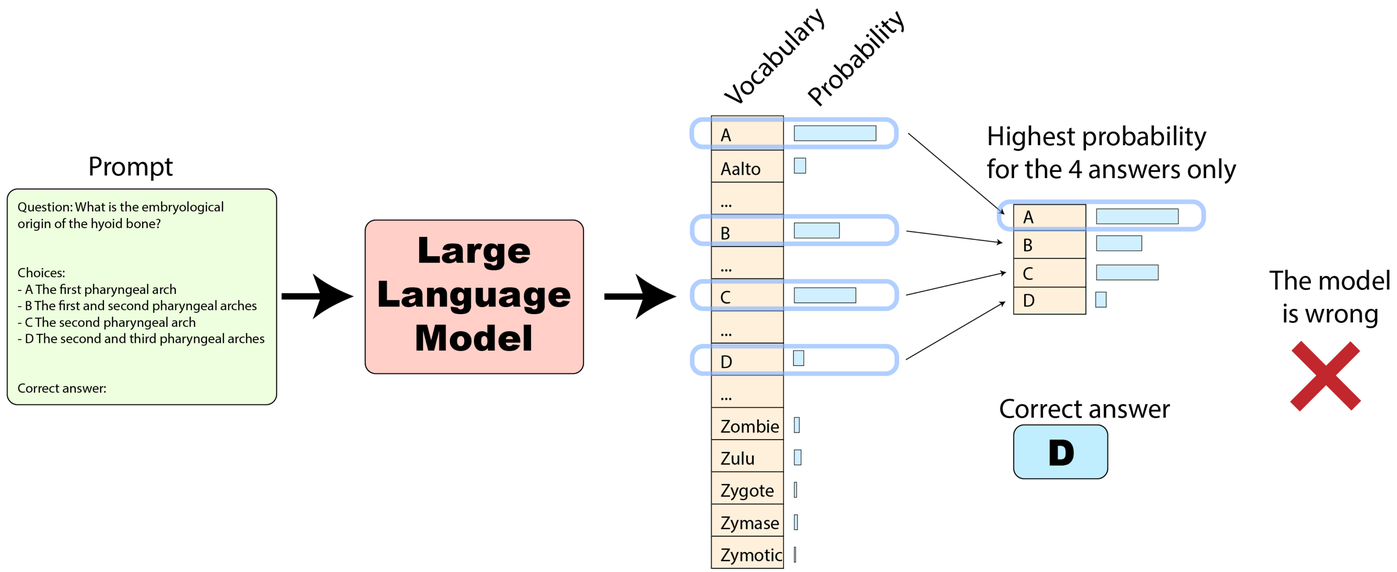


Diagram taken from https://huggingface.co/blog/open-llm-leaderboard-mmlu

Why Visualization Matters?

Making it easier to manage and interpret large volumes of experimental data

- Simplifies understanding complex evaluation data
- Enhances collaboration by sharing standardized dashboards
- Identifies trends and patterns for model improvement

- MLflow Overview – **used in the lab**
  - Open-source platform for managing machine learning experiments
  - Provides tools to log parameters, metrics, artifacts, and models in a unified interface
  - Supports tracking evaluations across multiple runs for consistent comparison

- Weights & Biases Overview
  - Comprehensive experiment tracking and visualization tool
  - Monitors and logs metrics in real-time for easier analysis
  - Offers customizable dashboards and hyperparameter sweeps
  - Facilitates collaboration by allowing teams to share and compare experiment runs

Acting on Evaluation Results – From Insights to Impact

Diagnose & Prioritize

Review Key Metrics
Root Cause Analysis

Define Actionable Next Steps

Fine-tuning & Data Augmentation
Prompt Engineering
Model Architecture & Hyperparameter Tuning

Implement Iterative Improvements

A/B Testing
Continuous Monitoring

Communicate & Collaborate

Share Findings
Integrate Feedback

Light Customization of LLMs

Unlocking the Power of LLMs: Why adding knowledge to LLMs matters?

A bag of techniques that unlocks the full potential of LLMs and Agentic AI

Bridge the Domain gap

Boost accuracy & Reduce hallucinations

Improve efficiency & Cost effectiveness

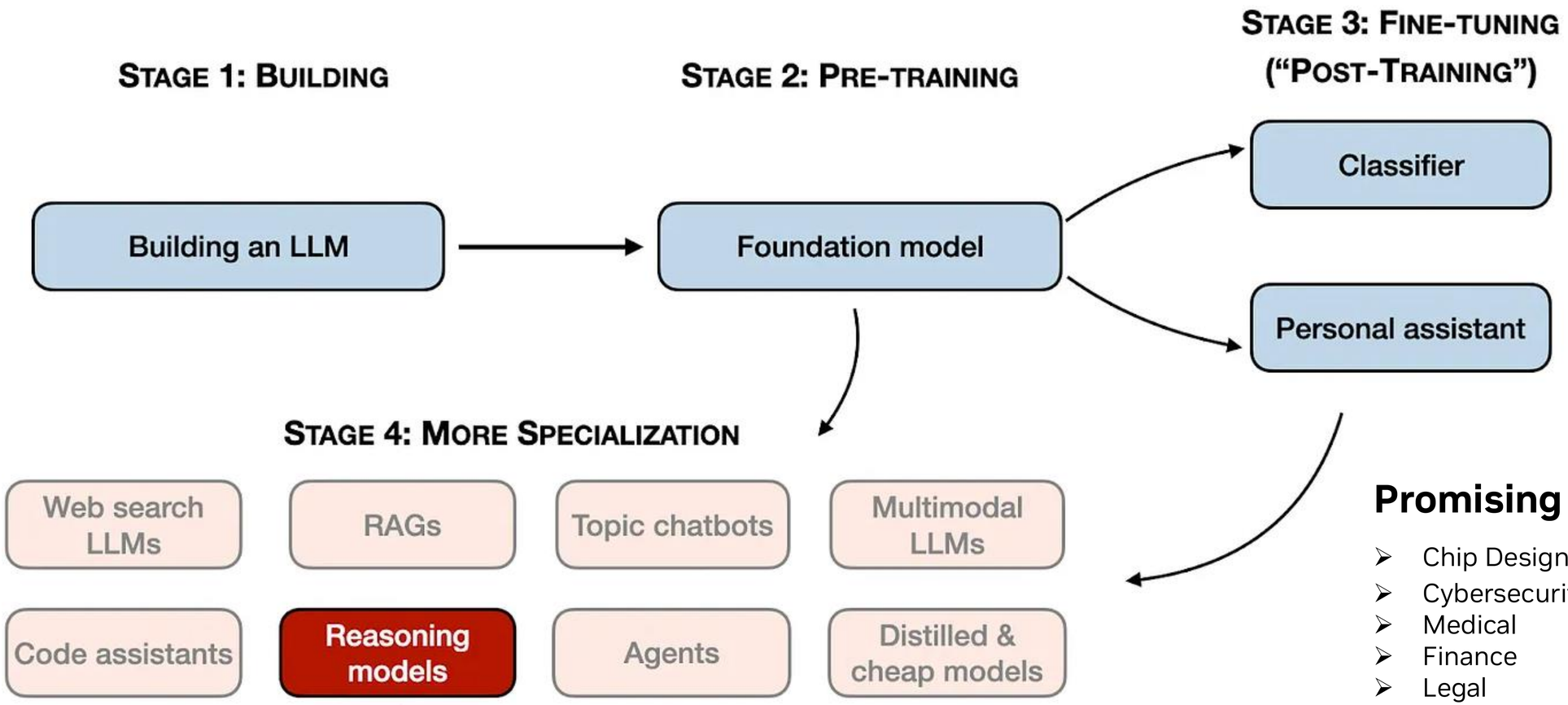
Enhance control & compliance

Accelerate time-to-value

Naturalize AI – tone, style, format, lingu

Adding Knowledge to LLMs

Methods for adapting large language models



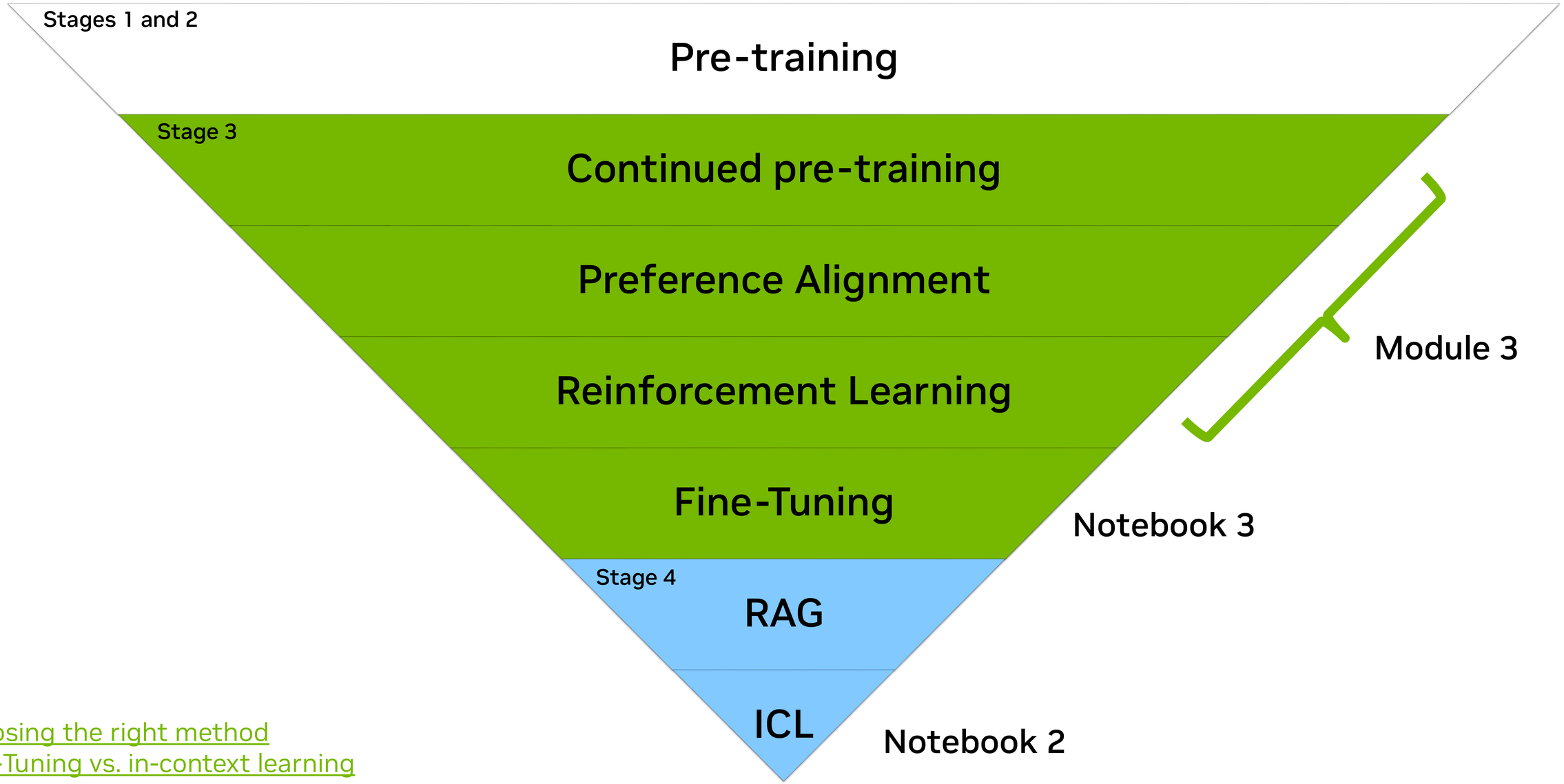
Promising Domains

- Chip Design
- Cybersecurity
- Medical
- Finance
- Legal
- Chemistry
- Low-resource Languages
- Vision Transformers And VLMs
- Sovereign LLMs

* Slide from Domain-Adaptive Pre-Training: Tailoring LLMs for Specialized Applications | GTC 25

Adding Knowledge to LLMs

Methods for adapting large language models



Choosing the right method
Fine-Tuning vs. in-context learning
Fine-tuning ~~vs.~~ and RAG

In-Context Learning (ICL)

- Zero-Shot
  - No examples; relies purely on model's pretraining
  - Fast ramp-up, but higher risk of off-topic or generic outputs
- Few-Shot
  - Embed a handful of input–output examples in the prompt
  - Guides the model's behavior without any parameter update

Feature	Zero-Shot Learning	In-Context Learning (ICL)
Definition	No task-specific examples provided	A few task-specific examples provided
Pros	• Quick, simple setup • No examples needed • Handles many tasks with minimal input	• Higher accuracy for new/complex tasks • Adapts to user needs • Clarifies task intent
Cons	• Lower accuracy on novel/complex tasks • Sensitive to prompt phrasing • May misinterpret ambiguous tasks	• Requires crafting examples • Longer prompts • Sensitive to example quality/diversity

The model’s ability to learn new tasks / patterns using ICL

Example:

- Suppose you want a language model to convert English numbers to French. You provide examples in your prompt:
  - 1: One -> Un 2: Two -> Deux 3: Three -> Trois 4: Four -> ?
- The model sees several examples and, "in context," predicts:
  - 4: Four -> Quatre
- Explanation:
  The model uses the examples in the prompt (the context) to infer the pattern and generate the correct answer, even though it hasn’t been explicitly trained on this task for this session.
- This is different from traditional training, where the model’s weights are updated—here, the model adapts to the task just by seeing a few examples.

Fine-Tuning Spectrum – From Full-Param to Lightweight Adapters

- Supervised Fine-Tuning (SFT)
  - Train all model parameters on labeled task data
  - Pros: highest task accuracy
  - Cons: expensive compute, large checkpoints
- Domain-Adaptive Fine-Tuning (DAFT)
  - Continued pretraining (e.g. MLM) on in-domain text (unlabeled or weakly labeled)
  - Pros: bridges style/vocab gaps
  - Cons: still full-param & compute heavy
- Trade-Offs at a Glance
  - Compute & storage ↑ vs. performance ↑
  - Iteration time ↑ vs. customization depth ↑

Deep Dive into Parameter-Efficient Fine-Tuning (PEFT)

- Core Principle
  - Keep backbone frozen; inject a small set of trainable parameters
  - Popular PEFT Methods
    – Adapters: small MLP “bottleneck” modules inserted between layers
    – LoRA: low-rank decomposition of weight updates
    – Prefix Tuning / P-Tuning: learn continuous prompts prepended to keys/values
    – BitFit: tune only bias terms
- Benefits
  – 1–2 orders of magnitude fewer trainable params
  – Faster training (less GPU/memory), easier checkpointing
  – Simpler version control (multiple adapters for different tasks)

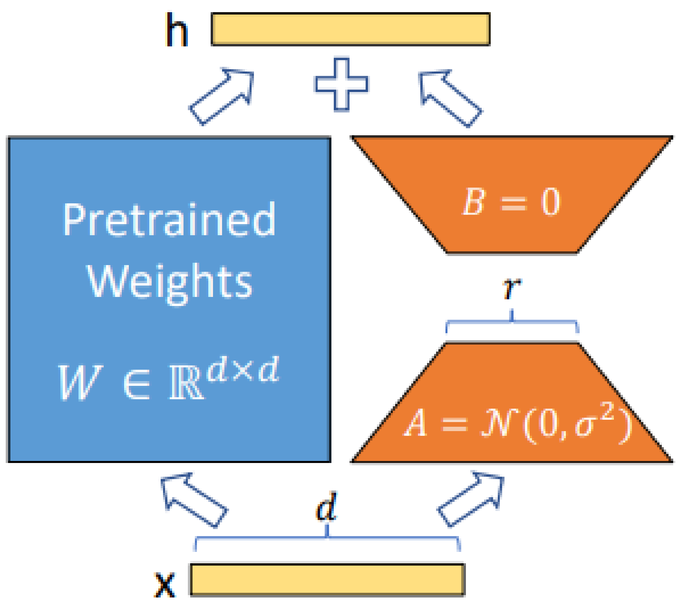


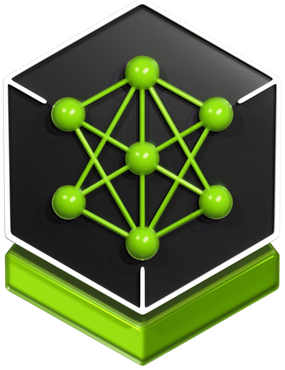
*Figure 1. Parameters in **A** and **B** represent the newly added information. Image credit: LoRA: Low-Rank Adaptation of Large Language Models*

Deep Dive into Parameter-Efficient Fine-Tuning (PEFT)

- Considerations & Best Practices
  – Choose PEFT method by task: LoRA for dense updates; adapters for modularity; prefix for prompt-like control
  – Typical trainable params: 0.1–1% of full model
  – Learning rate: often higher than SFT (e.g. 1e-4 to 1e-3)
  – Monitor convergence on a held-out set to avoid over-adaptering
- When to Use PEFT
  – Rapid prototyping across many tasks
  – Resource-constrained environments (edge, multi-tenant servers)
  – You need fine-tuned performance with minimal operational overhead

Objectives of Notebook 1

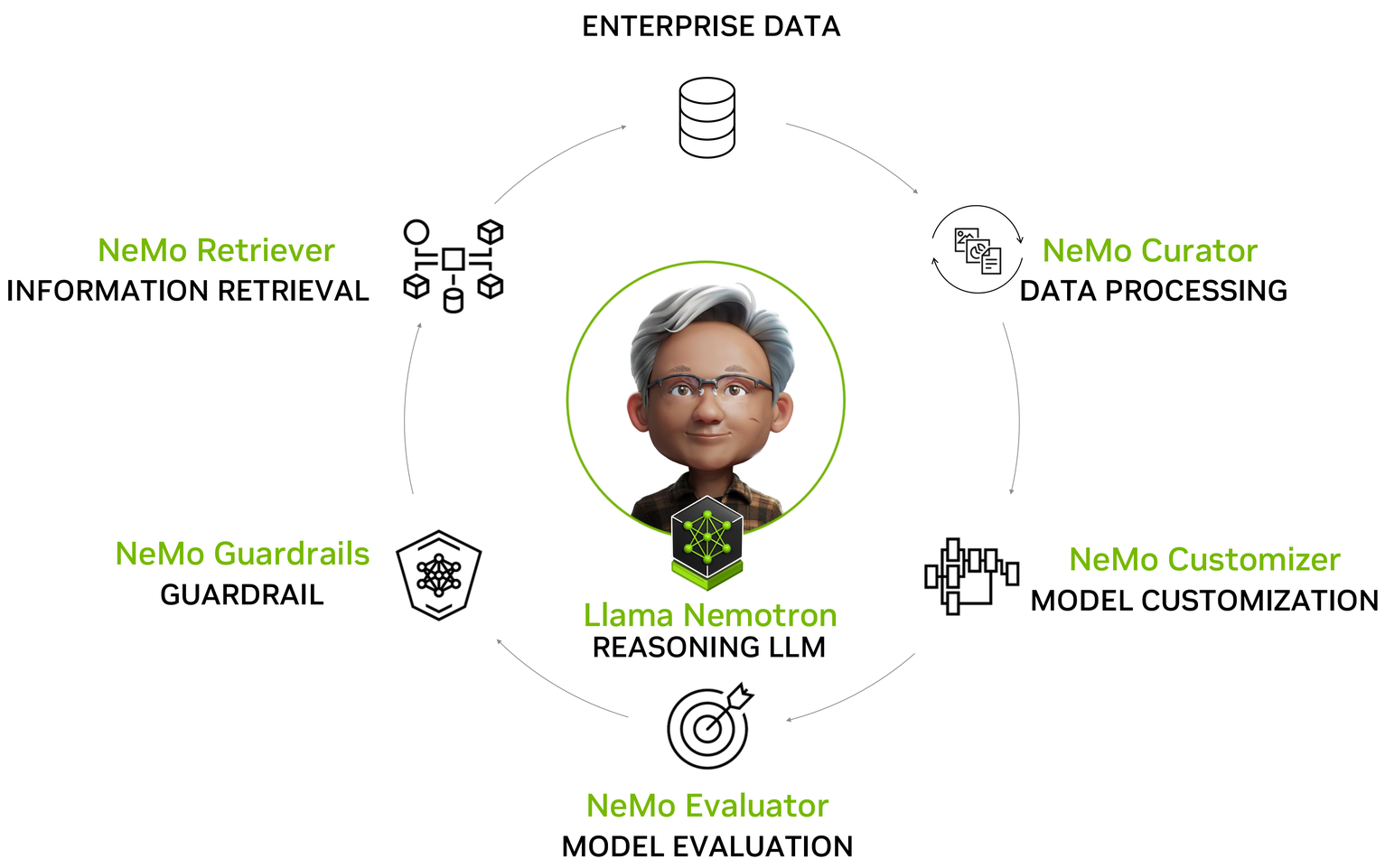
- Notebook 1:
  - Familiarize yourself with querying an LLM via curl and python.
  - Understand benchmark evaluation and evaluate on the Mathematics task gsm8k.
  - Perform LLM-as-a-judge evaluation.
  - Optional - Learn the principles of human evaluation and understand ELO ranking for the LLM arena



Notebooks 2 + 3

NVIDIA NeMo Microservices

End-to-End Platform with Modular Microservices To Scale AI Agent Optimizations



Easy to Operate | Boost Accuracy | Increase Efficiency | Enterprise-Grade | Run Anywhere

Enterprises Need Custom LLMs

Custom Models Provide Insightful Responses

Base Models Generate Generic Responses

Banking Chatbot

Hi. How can I help you?

How do I apply for a business loan?

U

To apply for a business loan, you'll typically need to provide financial statements, a business plan, and personal identification.

Custom Models Provide Business-Specific Answers

Banking Chatbot

Hi. How can I help you?

How do I apply for a business loan?

U

To apply for a business loan, visit our website's Business Banking section and fill out the application form. You'll need two years of financial statements, a business plan, and tax returns, with additional requirements for loans over $500,000.

Keep Agents Continuously Performing at Their Peak

NVIDIA NeMo Microservices Simplifies and Accelerates End-to-end Data Flywheel

Easy Data Flywheel Setup
Modular microservices, deployable with standard APIs

Broad Ecosystem Support
Integrated in popular open frameworks and AI software platforms

Enterprise-grade
Secure, stable, and supported software

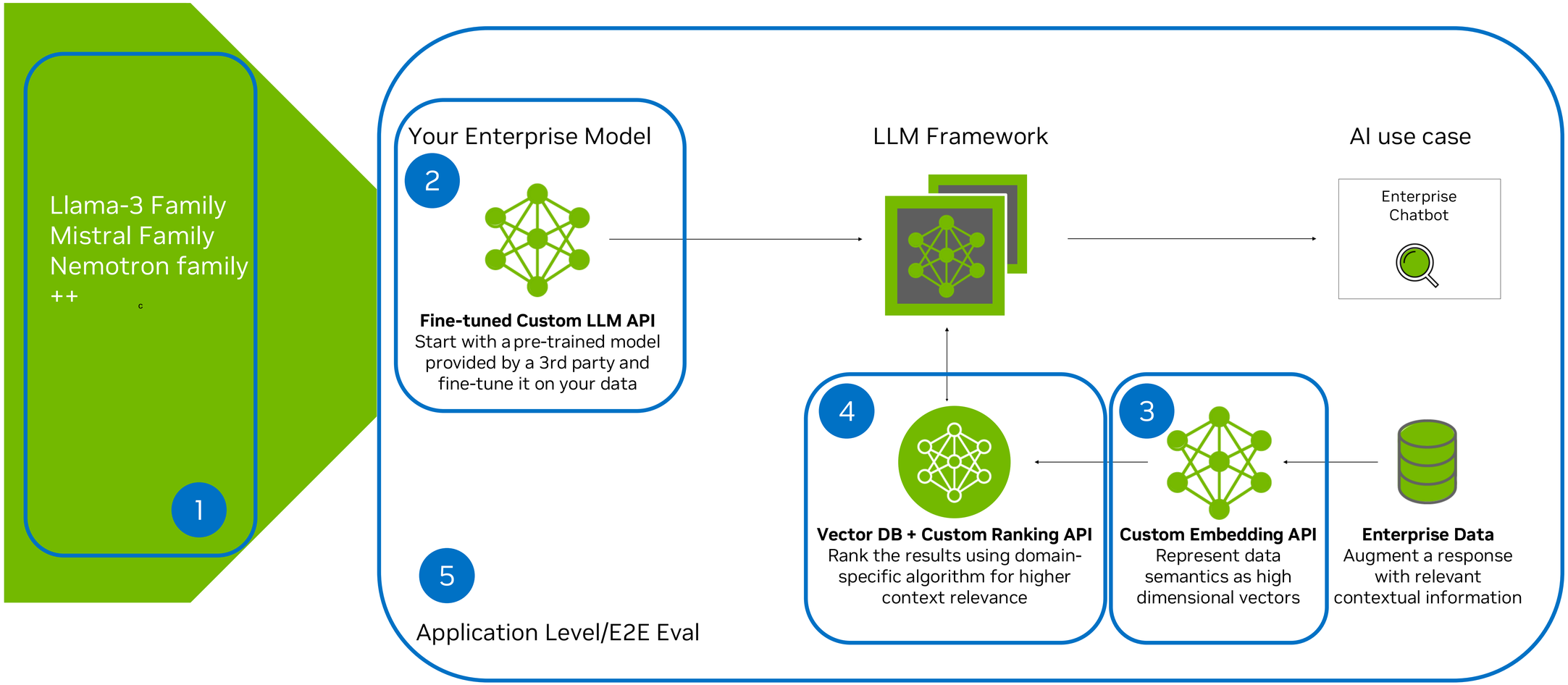
Streamlined Data Review
Centralized data collection, automatic insights, faster human review

Higher Accuracy
Generate better outputs with curation, customization, evals, and guardrails

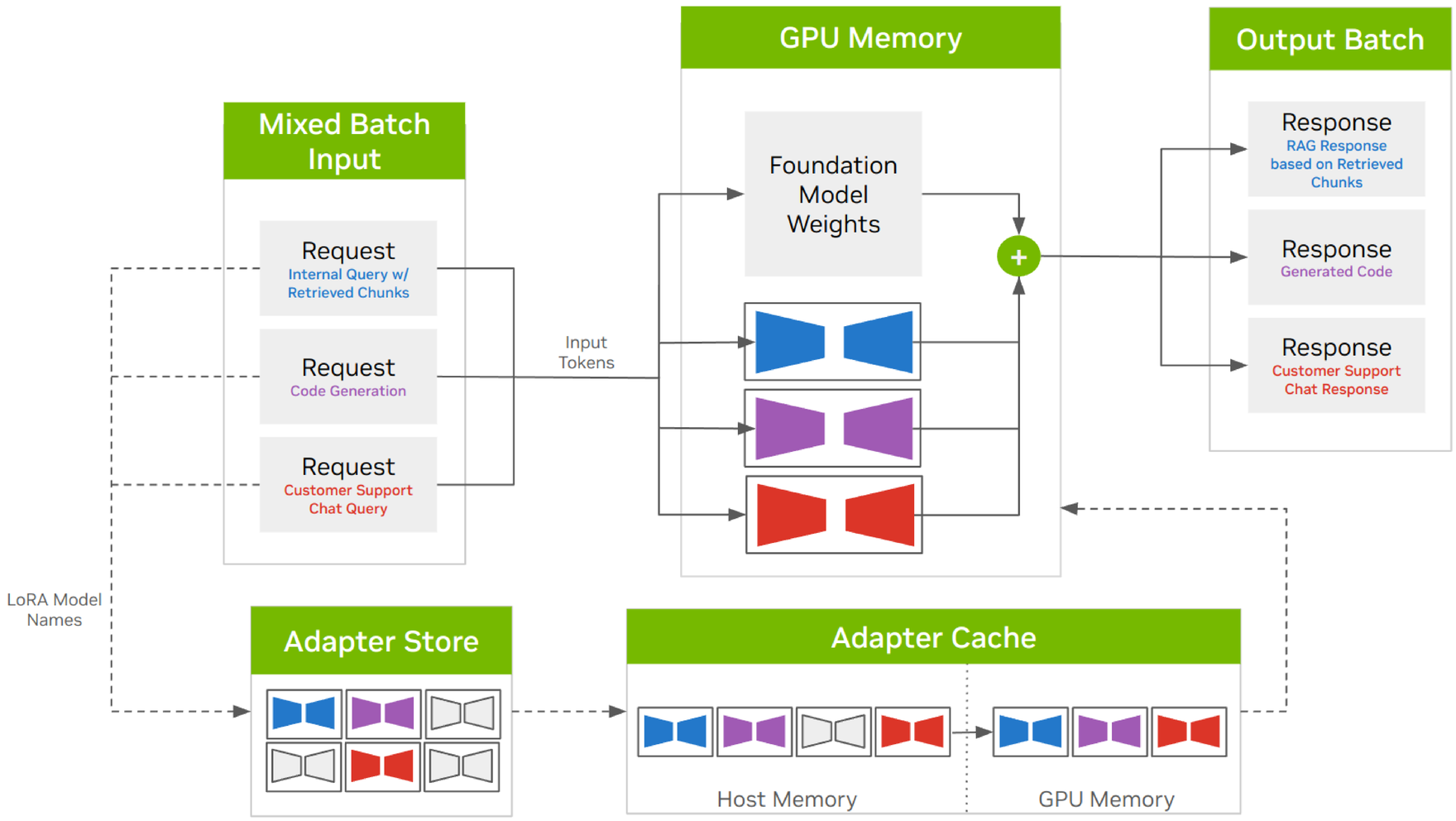
Run Anywhere
Provides higher security, privacy, and flexibility

Evaluate AI Pipelines and

NeMo Evaluator Supports RAG, LLMs, Custom LLMs, and Retrieval Models



Deploying LoRA adapters with NIM



Objectives of Notebook 2

- Notebook 2:
  - Start working with the Nemo Microservices
  - Format a custom dataset for evaluation and upload it to NeMo Data Store
  - Evaluate a locally deployed LLM in two modes – Zero-shot and Few-shots (ICL) using a custom dataset using similarity metrics ROUGE and LLM-As-A-Judge
  - Visualize the evaluation results in mlflow

Objectives of Notebook 3

- Notebook 3:
  - Learn how to fine-tune an LLM using Low-Rank Adaptation (LoRA) for domain-specific tasks
  - Understand the advantages of LoRA compared to zero-shot inference and in-context learning
  - Configure and launch training jobs with NeMo Customizer microservices
  - Compare the performance of fine-tuned models against baseline models
  - Evaluate the trade-offs between different adaptation approaches (zero-shot, ICL, and LoRA)

NVIDIA